

深圳市嘟嘟喵科技有限公司

smokefire 烟火 / 抽烟检测系统

部署手册

文档版本: V1.0

适用程序版本: 1.0.0

更新日期: 2026-08-03

目录

文档信息	3
1. 交付方式与系统要求	3
1.1 交付物	3
1.2 主机要求	3
2. 下载与校验	4
2.1 CPU 版	4
2.2 GPU 版	5
2.3 校验	5
3. CPU / GPU 部署	6
3.1 部署流程总览	6
3.2 Windows 前置准备	6
3.3 Windows 部署步骤	7
3.4 Linux 部署步骤	7
3.5 启动脚本的五阶段与预期输出	8
3.6 启停和日志	9
4. 视频流接入模式	9
5. 对接用户现有 go2rtc	10
5.1 对接原理	10
5.2 一次性配置	10
5.3 验证自动导入	10
6. 内置 go2rtc 与直接 RTSP	11
6.1 内置 go2rtc	11
6.2 直接 RTSP	11
7. 首次启动与验收	11
8. 配置与局域网访问	12

9. 日常运维	12
10. 备份、恢复与升级	13
11. 故障排查	13
12. 交付检查表	13
部署完成	13
视频流与业务验收	14

重要提示

本系统是视频 AI 辅助告警工具，不是经认证的火灾报警设备，不能替代法定消防设施、人工巡检、应急流程和现场复核。模型可能误报或漏报。

文档信息

项目	内容
文档名称	smokefire 部署手册
面向人员	实施工程师、系统管理员、现场运维人员
标准交付方式	Docker CPU / NVIDIA GPU 离线部署包，不含项目源码
支持宿主机	Windows 10/11、Windows Server、主流 x86_64 Linux
视频源	直接 RTSP、内置 go2rtc、用户现有 go2rtc 批量同步
默认访问地址	http://127.0.0.1:8600
项目地址	https://github.com/newtv-ai/smokefire

1. 交付方式与系统要求

1.1 交付物

GitHub Release v1.0.0 提供两套不含项目源码的离线交付：

- smokefire-deploy-1.0.0-cpu.zip：CPU 版完整包；
- smokefire-deploy-1.0.0-gpu.zip：GPU 版配置、权重、脚本和手册；
- GPU 镜像分卷 smokefire-images.tar.gz.partNN：因镜像较大，需与 GPU 包一起下载；
- SHA256SUMS-v1.0.0.txt：Release 资产校验值。

每套部署均包含 smokefire 与 go2rtc 的预构建 Docker 镜像、fire_smoke_v5.pt 与 smoking_v4.pt、Docker Compose 配置、Windows / Linux 脚本以及中英文三套 PDF。目标机不需要 Python、Git 或编译工具，部署完成后运行不依赖 GitHub。

1.2 主机要求

项目	CPU 版	GPU 版
处理器	x86_64，建议 4 核以上	x86_64，建议 4 核以上
内存	最低 8 GB，建议 16 GB	最低 16 GB
显卡	不需要	NVIDIA GPU，建议 8 GB 以上显存
驱动	-	主机驱动须支持交付镜像内的 CUDA 12.8 运行时
Docker	Engine / Desktop 24+，Compose v2	同左，并安装 NVIDIA Container Toolkit
磁盘	至少 20 GB，另计事件数据	至少 50 GB，另计事件数据

项目	CPU 版	GPU 版
摄像头	RTSP, 推荐 H.264	同左

CPU 版适合试点、小规模部署和无 NVIDIA GPU 的场景。GPU 版不以某个测试显卡型号承诺兼容性或承载路数；目标主机必须先确认驱动能够运行交付镜像内的 CUDA 12.8 运行时，再用真实码流完成启动、双模型加载、显存和持续运行验收。实际容量取决于显卡、驱动、编码、分辨率、检测间隔、录像和现场网络。

2. 下载与校验

仓库当前为 Public。用户无需登录 GitHub，即可进入以下仓库的 Releases 页面下载 **v1.0.0**：

<https://github.com/newtv-ai/smokefire>

Release 资产一览，按所选版本下载对应行：

资产	大小	CPU 版	GPU 版
smokefire-deploy-1.0.0-cpu.zip	约 913 MB	必需	-
smokefire-deploy-1.0.0-gpu.zip	约 90 MB	-	必需
smokefire-images.tar.gz.part01	1.80 GB	-	必需
smokefire-images.tar.gz.part02	1.80 GB	-	必需
smokefire-images.tar.gz.part03	1.16 GB	-	必需
SHA256SUMS-v1.0.0.txt	约 1 KB	建议	建议

CPU 包自带镜像，一个 zip 即可；GPU 镜像超过 GitHub 单文件上限，拆成三个分卷单独发布，因此 GPU 版必须下载"一个 zip + 三个分卷"，缺一个都无法导入。

安装路径建议使用不含空格和中文的目录，例如 Windows 用 `D:\smokefire`、Linux 用 `/opt/smokefire`。

2.1 CPU 版

- 1 打开 Releases 页面，展开 **v1.0.0** 的 Assets 列表。
- 2 下载 `smokefire-deploy-1.0.0-cpu.zip` 和 `SHA256SUMS-v1.0.0.txt`。
- 3 解压 zip 到目标目录。

解压后的目录结构：

```
D:\smokefire\  
docs\          手册 PDF（中英文各三本）  
images\        离线镜像（CPU 包自带）  
models\        两个模型权重  
.env.example    配置模板  
docker-compose.yml  
docker-compose.gpu.yml  
start.ps1      start.sh  
stop.ps1       stop.sh  
verify.ps1     verify.sh  
configure-go2rtc.ps1  configure-go2rtc.sh  
SHA256SUMS
```

2.2 GPU 版

- 1 下载 [smokefire-deploy-1.0.0-gpu.zip](#)。该包只含配置、模型、脚本和手册，不含镜像。
- 2 下载全部三个分卷 [smokefire-images.tar.gz.part01](#) / [part02](#) / [part03](#)，合计约 4.8 GB。
- 3 下载 [SHA256SUMS-v1.0.0.txt](#)。
- 4 解压 GPU 包到目标目录。
- 5 把三个分卷原样放进解压出来的 [images\](#) 目录。

分卷必须保持原文件名，不要改名、不要解压、不要自行合并。放好之后 [images\](#) 应该是：

```
D:\smokefire\images\  
README.txt  
smokefire-images.tar.gz.part01  
smokefire-images.tar.gz.part02  
smokefire-images.tar.gz.part03
```

启动脚本会按文件名顺序临时合并分卷、导入镜像，随后只删除自己生成的临时合并文件，不会删除原始分卷。合并期间 [images\](#) 需要额外约 4.8 GB 可用空间。

2.3 校验

不需要单独执行校验。 启动脚本的第一步就会自动校验交付文件，失败会直接停下并指出是哪个文件，所以正常部署只要跑一条命令（见第 3 章）。

自动校验的输出是一行汇总：

```
Verified 25 files.
```

CPU 包应校验 22 个文件，GPU 包 25 个（多出三个镜像分卷）。数量对不上说明有文件没放到位。

只有在想“先确认下载是否完整、暂时不启动服务”的时候，才需要手工跑一次校验：

```
powershell -ExecutionPolicy Bypass -File .\verify.ps1
```

```
chmod +x verify.sh start.sh stop.sh configure-go2rtc.sh  
./verify.sh
```

模型固定校验值：

文件	大小	SHA-256
models/fire_smoke_v5.pt	44,014,233 字节	b5248d55c5d90e341bc4e537887d4bce4b9af1bd0330ddd87825294750a1e216
models/smoking_v4.pt	44,025,689 字节	0ea7b66c2105f1898f56f828b18ffb01934fb1f47a795192e8867ae8fcb128dd

任一校验失败都不要继续部署，应重新下载对应文件。GPU 分卷下载中断是最常见的失败原因，只需重下报错的那个分卷，不必重下整包。校验通过后再进入第 3 章。

3. CPU / GPU 部署

3.1 部署流程总览

不分 CPU / GPU、不分 Windows / Linux，主线都是两步：装好 Docker，跑一次启动脚本。校验、导入镜像、起容器、等就绪都由启动脚本一次做完，内部分五个阶段，每个阶段都会在终端打印 [N/5]。

序号	动作	命令	预计耗时
1	安装并启动 Docker	见 3.2 / 3.4	10-30 分钟，含下载
2	运行启动脚本（含自动校验）	<code>start.ps1</code> / <code>start.sh</code>	CPU 约 5 分钟，GPU 约 15 分钟
3	打开 Web 页面确认就绪	浏览器访问 http://127.0.0.1:8600	1 分钟
4	选择视频接入模式	<code>configure-go2rtc.*</code> ，见第 4 章	5 分钟起

首次启动的绝大部分时间花在离线导入 Docker 镜像上，属于正常现象。导入完成后的镜像占用：

镜像	占用
smokefire:1.0.0-cpu	约 3.1 GB
smokefire:1.0.0-gpu	约 13.3 GB
alexxit/go2rtc:1.9.9	约 0.4 GB

GPU 版首次启动的磁盘峰值为：分卷约 4.8 GB + 临时合并文件约 4.8 GB + 镜像约 13.7 GB。Docker 数据目录所在磁盘应预留 25 GB 以上可用空间，事件录像另计。

3.2 Windows 前置准备

- 1
- 安装 Docker Desktop，安装过程保持默认的 WSL 2 后端。
- 2
- 启动 Docker Desktop，等待界面左下角状态变成 Running。
- 3
- 确认处于 Linux 容器模式。托盘图标右键菜单若显示 "Switch to Windows containers..."，说明当前已经是 Linux 容器，不要点。
- 4
- 在 PowerShell 中验证：

```
docker version
docker compose version
```

`docker version` 必须同时输出 Client 和 Server 两段。只有 Client 段说明引擎没起来，回到第 2 步等 Docker Desktop 变成 Running 再继续。

GPU 版还需要三步：

- 1 安装或更新 NVIDIA 显卡驱动，驱动须支持交付镜像内的 CUDA 12.8 运行时。
- 2 在 Docker Desktop 的 Settings → Resources 中确认 GPU 支持已启用，WSL 2 后端默认启用。
- 3 在宿主机验证显卡可见：

```
nvidia-smi
```

`nvidia-smi` 必须列出显卡型号和驱动版本。这一步不通过就不要继续，先解决驱动问题。容器内的 CUDA 验证需要镜像已导入，放在启动脚本跑完之后，见 3.5。

3.3 Windows 部署步骤

在已经打开的 PowerShell 窗口里执行下面的命令。不要双击脚本，也不要右键“使用 PowerShell 运行”：那种方式下脚本一旦报错，窗口会立刻关闭，错误信息来不及看，现象就是“跑到一半自己退出了”。

```
cd D:\smokefire
powershell -ExecutionPolicy Bypass -File .\start.ps1
```

就这一条。校验交付文件是启动脚本的第一步，不用再单独跑 `verify.ps1`。

`-ExecutionPolicy Bypass` 只对这一次调用生效，不会修改系统执行策略。

脚本全程会把输出写进部署目录下的 `start-log.txt`。出任何问题，把这一个文件发给技术支持就够了，不用截图、也不用回忆屏幕上滚过什么。

脚本结束时停在“按回车键关闭”，所以即使双击运行，窗口也不会一闪而过。写自动化脚本调用时加 `-NoPause` 跳过这个停顿。

脚本任何一步失败都会打印原因并停止，不会带着半个环境继续往下跑。中途停下时，先看终端里最后出现的 `[N/5]`，再对照 3.5 定位。

3.4 Linux 部署步骤

安装 Docker Engine 与 Compose v2；GPU 版还要安装 NVIDIA 驱动和 NVIDIA Container Toolkit。验证：

```
docker version
docker compose version
nvidia-smi          # 仅 GPU 版
```

部署：


```
cd /opt/smokefire
chmod +x verify.sh start.sh stop.sh configure-go2rtc.sh
./start.sh
```

`chmod` 里仍要带上 `verify.sh`，因为启动脚本会调用它，但不需要单独执行。

当前用户不在 `docker` 组时，上述 `docker` 相关命令需要 `sudo`。阶段输出与 3.5 一致。

3.5 启动脚本的五阶段与预期输出

`start.ps1` 与 `start.sh` 行为一致，依次打印 [1/5] 到 [5/5]。下面是每个阶段的作用、正常输出和卡住时的处理。

[1/5] 校验交付文件

```
[1/5] Verifying deployment files...
Verified 25 files.
```

脚本自动完成校验，不需要另外运行。CPU 包 22 个文件，GPU 包 25 个。停在这里说明文件缺失或校验值不符，脚本会指出是哪个文件，按 2.3 重新下载它。

[2/5] 检查 Docker

```
[2/5] Checking Docker...
```

检查 `docker info` 与 `docker compose version`。停在这里说明 Docker 引擎没有启动，或者 Compose v2 缺失，脚本会明确指出是哪一种。

首次运行会在这一步之后多打印一行：

```
Created .env from .env.example
```

表示已按模板生成本次部署的 `.env`。GPU 包的模板里已经写好 `SMOKEFIRE_IMAGE=smokefire:1.0.0-gpu` 和 `COMPOSE_FILE=docker-compose.yml|docker-compose.gpu.yml`，不需要手工改。

[3/5] 离线导入镜像

```
[3/5] Importing the prebuilt smokefire and go2rtc images when needed...
Assembling smokefire-images.tar.gz.part01...
Assembling smokefire-images.tar.gz.part02...
Assembling smokefire-images.tar.gz.part03...
Loaded image: smokefire:1.0.0-gpu
Loaded image: alexxit/go2rtc:1.9.9
```

耗时最长的一步。GPU 版要先合并分卷再导入，机械硬盘上可能需要十几分钟，期间没有进度条属于正常，不要中断。镜像已经存在时这一步只打印一行 `Images ... are already available.` 并跳过。

[4/5] 启动容器

```
[4/5] Starting smokefire...
Container smokefire-go2rtc-1 Started
Container smokefire-smokefire-1 Started
```

先起 go2rtc，等它健康检查通过后再起 smokefire，顺序由 compose 的依赖关系保证。

[5/5] 等待就绪

```
[5/5] Waiting for readiness (up to 5 minutes)...
smokefire 1.0.0 is ready: http://127.0.0.1:8600
```

每 5 秒轮询一次 `/api/health/ready`，最多等 5 分钟。首次启动要加载两个模型，等一两分钟属于正常。超时脚本会自动打印容器状态和最近 200 行日志再退出，把这段输出交给技术支持即可定位。

GPU 版在启动脚本跑完之后，补做一次容器内验证：

```
docker run --rm --gpus all smokefire:1.0.0-gpu python -c "import torch;
print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

应输出 `True` 和显卡型号。输出 `False` 说明容器没拿到 GPU，回到 3.2 检查驱动和 Docker Desktop 的 GPU 支持。不要带着“实际跑在 CPU 上”的状态交付 GPU 版。

3.6 启停和日志

```
docker compose ps
docker compose logs -f smokefire
```

Windows 停止：

```
.\stop.ps1
```

Linux 停止：

```
./stop.sh
```

停止脚本不会删除业务数据。不要执行 `docker compose down -v`，否则会删除命名卷。

4. 视频流接入模式

三种模式按现场情况三选一：

模式	适用情况	配置量	数据流
A. 直接 RTSP	少量摄像头，没有 go2rtc	每路填写一次	摄像头 / NVR -> smokefire
B. 内置 go2rtc	在 smokefire 中管理摄像头，希望检测和录像共享上游连接	每路在 smokefire 添加一次	摄像头 -> 内置 go2rtc -> 检测与录像
C. 用户现有 go2rtc	现场已经用 go2rtc 管理全部视频流	只填一次 API 和 RTSP 基址	现有 go2rtc -> 自动导入 -> smokefire 基址

已有 go2rtc 的项目优先使用模式 C。不要再把每路 RTSP 手工复制进 smokefire。

切换模式使用交付包内的配置脚本；脚本只修改当前部署目录的 `.env`。若服务已运行，脚本会重建 smokefire 容器使配置生效，业务数据仍保留。

5. 对接用户现有 go2rtc

5.1 对接原理

smokefire 沿用上游系统的批量同步方式：

```
用户 go2rtc GET /api/streams
  |
  v
smokefire 自动同步主流名称与状态
  |
  v
rtsp://用户-go2rtc:8554/<编码后的流名称>
  |
  v
AI 检测、录像、预览与事件
```

同步器只读取用户 go2rtc 的 `GET /api/streams`，不会调用其新增、修改或删除接口。首次同步会自动注册全部主流码流；发现同名的 `_sub`、`-sub` 或 `_sd` 子码流时，优先保留主流码流，避免重复摄像头。后续按默认 300 秒同步：新增流自动加入，消失流自动停用，恢复后自动启用。

5.2 一次性配置

现有 go2rtc 与 smokefire 在同一台宿主机时，容器内不要写 `127.0.0.1`，应使用 `host.docker.internal`：

```
.\configure-go2rtc.ps1 -Mode upstream `
  -Api http://host.docker.internal:1984 `
  -Rtsp rtsp://host.docker.internal:8554
```

Linux：

```
./configure-go2rtc.sh upstream \
  http://host.docker.internal:1984 \
  rtsp://host.docker.internal:8554
```

若 go2rtc 位于另一台局域网主机，把主机名替换为其内网 IP：

```
API http://192.168.10.20:1984
RTSP rtsp://192.168.10.20:8554
```

对应 `.env` 为：

```
SMOKEFIRE_GO2RTC_ENABLED=false
SMOKEFIRE_UPSTREAM_GO2RTC_ENABLED=true
SMOKEFIRE_UPSTREAM_GO2RTC_API=http://192.168.10.20:1984
SMOKEFIRE_UPSTREAM_GO2RTC_RTSP=rtsp://192.168.10.20:8554
SMOKEFIRE_UPSTREAM_GO2RTC_SYNC_INTERVAL_SEC=300
```

5.3 验证自动导入

- 1 在用户 go2rtc 打开 `http://<go2rtc-host>:1984/api/streams`，确认返回流列表。
- 2 运行配置脚本并启动 smokefire。
- 3 打开“摄像头管理”，确认主码流自动出现并带“上游”标识。
- 4 确认匹配的子码流未重复创建，摄像头数量与预期一致。
- 5 检查至少一条预览、一次 AI 检测和一段事件录像。
- 6 临时增加一条测试流，确认下一次同步后自动加入；移除后自动停用，恢复后自动启用。

用户现有 go2rtc 的流名称应保持唯一、稳定。自动导入的源地址由同步器管理，不要在界面重复创建同名手工摄像头。

6. 内置 go2rtc 与直接 RTSP

6.1 内置 go2rtc

适用于用户没有现成 go2rtc、但希望减少摄像头上游连接数的场景：

```
.\configure-go2rtc.ps1 -Mode builtin
```

```
./configure-go2rtc.sh builtin
```

之后仍在 smokefire 的“摄像头管理”逐路添加 RTSP。系统自动为每路建立 `sf-camN` 内部别名，检测和录像都从内置 go2rtc 取流。smokefire 只管理这些内部别名，不影响外部系统。

6.2 直接 RTSP

```
.\configure-go2rtc.ps1 -Mode direct
```

```
./configure-go2rtc.sh direct
```

然后在界面逐路填写摄像头或 NVR 提供的 RTSP 地址。该模式最简单，但检测与录像可能分别建立上游会话，路数增加时优先改用 go2rtc。

7. 首次启动与验收

在服务主机打开 `http://127.0.0.1:8600`。首次加载两个模型可能需要数十秒至数分钟。

地址	用途	期望结果
<code>/api/health/live</code>	进程是否存活	HTTP 200
<code>/api/health/ready</code>	模型、调度、录像、磁盘和上游同步是否就绪	HTTP 200
<code>/api/status</code>	运行指标和通知统计	返回 JSON

最小验收：

- 1 预览墙、事件中心、摄像头管理均可打开。
 - 2 按所选模式接入视频；现有 go2rtc 模式必须验证批量自动导入。
 - 3 摄像头最终显示“在线”，名称、画面和时间正确。
 - 4 使用授权测试素材或现场可控方式触发烟火和抽烟模型告警。
 - 5 事件中心能看到快照、类别、置信度、时间和录像。
 - 6 验证确认属实、标记误报、断流恢复和服务重启后数据保留。
 - 7 在计划路数下持续运行，记录吞吐、告警延迟、显存/CPU、磁盘和网络。
- 部署成功不等于正式验收完成；模型边界和样本建议见《模型能力实测手册》。

8. 配置与局域网访问

默认 Web 端口绑定到 127.0.0.1:8600。如需从局域网其他电脑访问，由实施人员在 .env 中设置绑定地址和允许的访问主机名，再重启服务。

配置项	默认值	说明
SMOKEFIRE_IMAGE	CPU / GPU 包内已设定	当前应用镜像
SMOKEFIRE_ALLOWED_HOSTS	localhost,127.0.0.1	允许访问 Web 的 Host
SMOKEFIRE_GO2RTC_ENABLED	false	使用内置 go2rtc
SMOKEFIRE_UPSTREAM_GO2RTC_ENABLED	false	同步用户现有 go2rtc
SMOKEFIRE_UPSTREAM_GO2RTC_API	空	现有 go2rtc API 基址
SMOKEFIRE_UPSTREAM_GO2RTC_RTSP	空	现有 go2rtc RTSP 基址
SMOKEFIRE_INFER_WORKERS	4	推理 worker 数
SMOKEFIRE_INFER_BATCH_MAX	8	最大推理批次
SMOKEFIRE_STOP_GRACE_PERIOD	180s	停机宽限时间

摄像头凭据可能出现在 RTSP 地址中。限制部署目录、备份和终端历史的访问范围，交付截图和工单中不要暴露完整凭据。

9. 日常运维

```
docker compose ps
docker compose logs --tail 200 smokefire
curl http://127.0.0.1:8600/api/health/ready
```

每天确认摄像头在线、上游同步最近一次成功、预览更新、最新事件可回放、通知队列正常、磁盘没有达到高水位。已有 go2rtc 模式下，还要对比 go2rtc 主码流数量与 smokefire 的“上游”摄像头数量。

10. 备份、恢复与升级

业务数据存放在 `smokefire-data` 命名卷。备份前停止服务，记录当前版本，并复制完整 `/app/data`；至少覆盖摄像头配置、SQLite 数据库、事件录像、快照、误报反馈和通知队列。用户现有 `go2rtc` 的配置由用户系统自行备份，不在 `smokefire` 数据卷内。

恢复：恢复同一版本对应的完整数据目录，启动后检查健康状态、摄像头、历史事件、录像、反馈和上游同步。

升级：先验证备份可读，再下载并校验新版本包，停止旧版本，在新目录启动并完成最小验收；异常时恢复旧镜像和升级前备份。

11. 故障排查

现象	优先检查	处理建议
Docker 命令不可用	Docker Desktop / Engine	启动引擎后重试 <code>docker version</code>
启动脚本没跑完就退出	部署目录下的 <code>start-log.txt</code>	该文件记录了全过程，直接发给技术支持
不确定卡在哪一步	终端里最后一行 <code>[N/5]</code> ，或 <code>start-log.txt</code>	对照 3.5 逐阶段定位
GPU 容器不可用	驱动、Container Toolkit、Docker GPU 支持	先通过 <code>nvidia-smi</code> 和容器 CUDA 验证
启动时报镜像分卷缺失	<code>images/</code> 、文件名、SHA-256	下载全部分卷并保持原名
<code>ready</code> 长时间不通过	日志、模型、磁盘、上游同步	<code>docker compose logs --tail 300 smokefire</code>
现有 <code>go2rtc</code> 没有自动导入	API 地址、容器到 1984 端口、 <code>/api/streams</code>	同机使用 <code>host.docker.internal</code> ，异机使用内网 IP
导入数量少于 <code>go2rtc</code>	主/子码流命名	<code>_sub</code> 、 <code>-sub</code> 、 <code>_sd</code> 与主码流匹配时会去重
上游摄像头变成停用	<code>go2rtc</code> 中流消失或同步失败	恢复该流；下次同步后自动启用
摄像头一直重连	RTSP 8554、名称编码、源流状态	在部署主机验证同一 <code>go2rtc</code> RTSP 地址
告警过多或没有告警	阈值、目标像素、画面干扰	用同一机位正负样本复测，不要盲目降阈值
事件没有录像	磁盘、录像预热、源流稳定性	检查磁盘状态和 FFmpeg 日志

12. 交付检查表

部署完成

- ☐ 已选择并校验 CPU 或 GPU 的完整离线交付。
- ☐ `smokefire`、`go2rtc` 和两个模型的校验均通过。
- ☐ 启动、停止、重启和主机重启恢复均验证通过。
- ☐ `/api/health/live` 与 `/api/health/ready` 返回正常。
- ☐ 已记录安装目录、数据卷、备份位置和 1.0.0 版本。
- ☐ 仓库保持 Public，用户可匿名打开仓库、Release 页面并下载校验文件。

视频流与业务验收

- ☐ 已明确使用直接 RTSP、内置 go2rtc 或用户现有 go2rtc。
- ☐ 现有 go2rtc 场景已验证一次配置、主码流批量导入、增删和恢复同步。
- ☐ 计划摄像头名称、时间、画面、检测和录像正确。
- ☐ 烟火与抽烟模型均用目标机位正负样本完成验证。
- ☐ 告警快照、录像、确认属实、误报归档和反馈导出正常。
- ☐ 已按计划路数验证容量、断流恢复和连续运行。
- ☐ 用户已收到《使用操作手册》和《模型能力实测手册》并完成交接培训。

编制单位：深圳市嘟嘟喵科技有限公司 **项目地址：**<https://github.com/newtv-ai/smokefire>